

Dispositivo IoT Antirobos

Alvarez Santiago Javier¹, Orellano Nazareno Asaf, Raimo Lugo Kevin¹, Altamirano
Fabrizio Augusto¹, Da Silva Lautaro Nahuel¹
45738927, 42562943, 43915535, 44161995, 42816815
Lunes, 4

¹Universidad Nacional de La Matanza,
Departamento de Ingeniería e Investigaciones Tecnológicas,
Florencio Varela 1903 - San Justo, Argentina

Resumen. El presente trabajo expone el diseño y desarrollo de un sistema IoT de seguridad orientado a la protección de equipaje personal. La solución consta de un dispositivo embebido basado en el microcontrolador ESP32 y una aplicación móvil nativa desarrollada en Android Studio mediante Jetpack Compose. Utiliza un SO de tiempo real FreeRTOS y una Máquina de Estados Finitos (FSM) para coordinar de forma concurrente sensores de contacto con los actuadores. La interacción con la aplicación móvil se implementa bajo una arquitectura asincrónica conducida por eventos mediante el protocolo MQTT protegido por semáforos de exclusión mutua. El sistema integra una infraestructura con Mosquitto, Node-RED, Telegraf e InfluxDB para la auditoría y persistencia de series temporales de datos, estos son consumidos de forma asincrónica por la aplicación Android para la visualización del historial de alertas. Los resultados demuestran un ecosistema tolerante a fallos y capaz de configurar sus parámetros en tiempo real.

Palabras claves: Sistemas de Tiempo Real, FreeRTOS, MQTT, InfluxDB, Exclusión Mutua

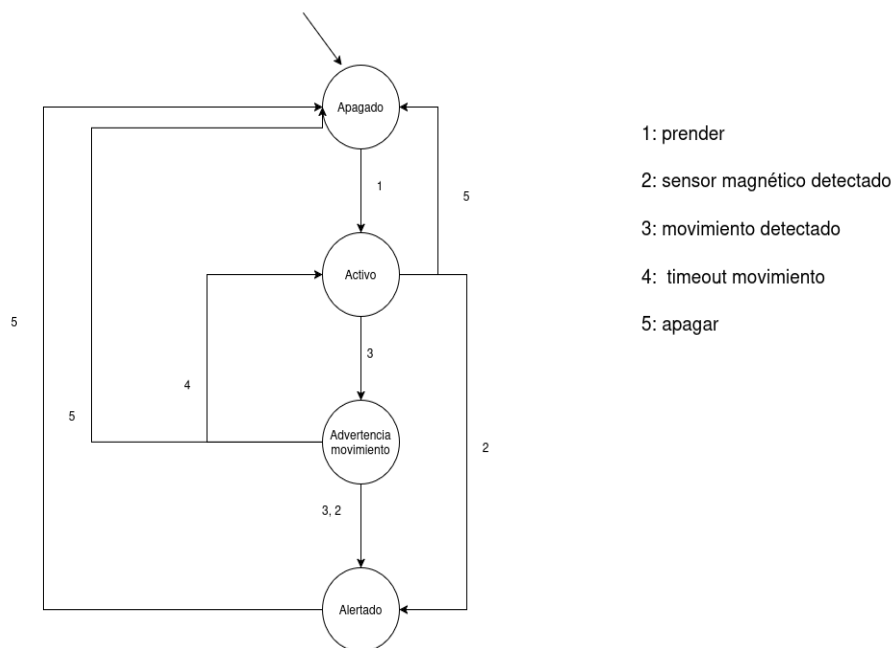
1 Introducción

- La seguridad de los efectos personales en espacios públicos representa una problemática vigente en entornos urbanos. Las soluciones convencionales suelen depender de barreras mecánicas pasivas que no alertan al usuario de forma temprana ante un intento de hurto o manipulación indebida.
- Este proyecto propone un ecosistema IoT integrado. El núcleo del hardware reside en un chip ESP32 que orquesta de forma proactiva la detección de amenazas mediante procesamiento perimetral (Edge Computing). Mediante el uso de FreeRTOS, se consolida un entorno multitarea capaz de evaluar de manera simultánea sacudidas bruscas y aperturas físicas, mientras mantiene un canal de comunicación asincrónico y bidireccional con una aplicación móvil.
- La utilidad principal radica en brindar al usuario control total sobre la seguridad de su mochila o bolso. Esto se logra permitiendo el armado/desarmado remoto,

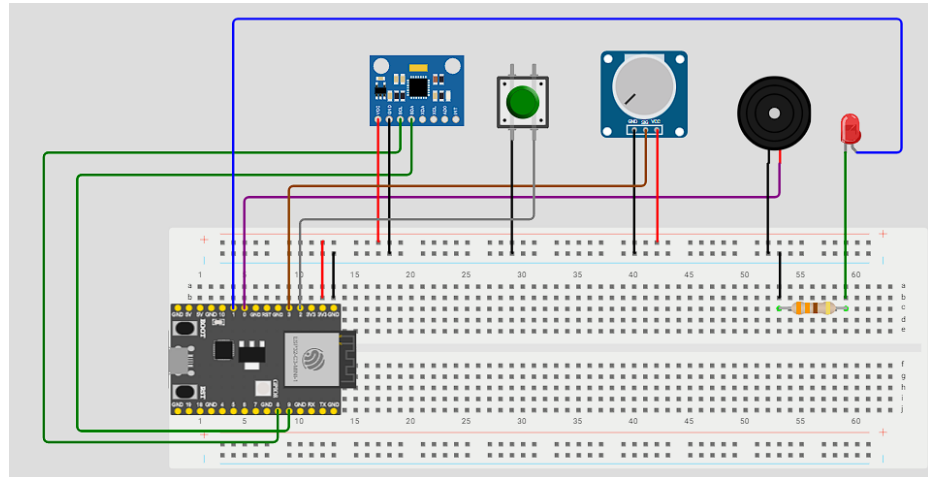
la calibración fina de la sensibilidad de los sensores de movimiento y la personalización de las respuestas auditivas de alerta ante intrusiones. Adicionalmente, con el propósito de auditoría y análisis forense de eventos históricos, el ecosistema incorpora una capa de persistencia distribuida de series temporales. Los eventos de alerta y métricas inerciales recolectados por el dispositivo embebido se centralizan en tiempo real para su posterior consulta y visualización asincrónica desde la interfaz móvil, transformando el dispositivo en un nodo IoT completo, auditable y tolerante a fallos.

2 Desarrollo

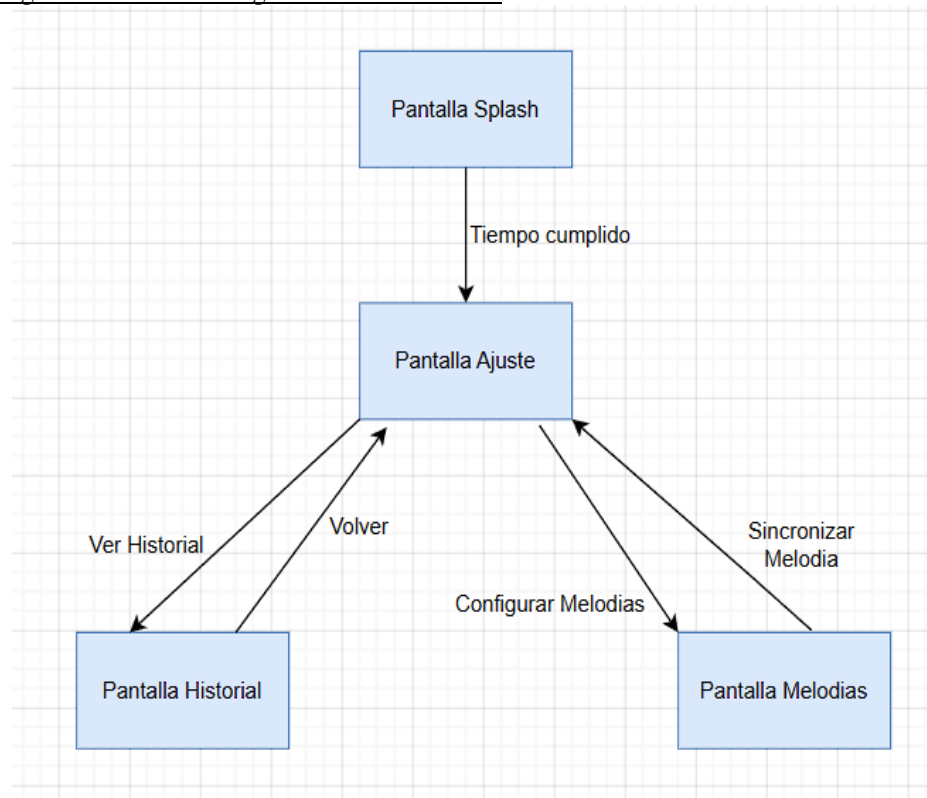
- [Link GitHub \(Código Fuente Completo Android y ESP32\):](https://github.com/UNLAM-SOA/2026-SOA-Q1-L4)
<https://github.com/UNLAM-SOA/2026-SOA-Q1-L4>
- [Link Wowki:](https://wokwi.com/projects/463137216781606913) <https://wokwi.com/projects/463137216781606913>
- [Diagrama de Máquina de Estados](#)



- [Circuito Wokwi](#)



- Diagrama funcional/navegación de las Actividades



- Manual de usuario:

1. El usuario toca el interruptor principal para encender la alarma (montar el dispositivo).
Se habilitará una barra deslizante (*Slider*) que va de 0 a 100 para regular nivel de Sensibilidad. Si el usuario se encuentra en un lugar con mucho movimiento

(como un colectivo o el subte), desliza la barra hacia un valor bajo para evitar falsas alarmas. Si deja la mochila quieta en una biblioteca o café, la desliza hacia un valor alto para máxima protección.

Una vez elegido el nivel, presiona el botón para establecer la sensibilidad

Al activar el interruptor desde el celular, el cerebro de la mochila (ESP32) activa y calibra los sensores.

Al enviar la sensibilidad, el Acelerómetro cambia de forma inmediata el umbral de detección de sacudidas en su memoria interna.

Si la mochila recibe un golpe o movimiento brusco que supere el nivel seleccionado, el LED indicador rojo de la placa se encenderá de inmediato como advertencia visual. El sistema vigilará el equipaje con alta prioridad durante un tiempo.

2. Pantalla de Configuración de Melodías (Personalización del Sonido)

Esta pantalla permite al usuario elegir qué música quiere que suene por el parlante de la mochila en caso de que intenten robarle.

El usuario presiona el botón para configurar melodías de Alarma para acceder a esta sección.

La pantalla le mostrará una lista de opciones con botones circulares (RadioButtons) para elegir entre 4 canciones disponibles: Star Wars (Marcha Imperial), Nokia Clásico, Super Mario Bros o Tetris.

El usuario selecciona la melodía de su preferencia y presiona el botón para sincronizar la melodía. La aplicación guardará la configuración.

Si un intruso abre el cierre de la mochila mientras está activa (lo cual es detectado físicamente por el Sensor) el sistema disparará la alarma definitiva.

En ese instante, el Zumbador (Buzzer) de la mochila empezará a sonar de forma ininterrumpida reproduciendo exactamente la canción que el usuario eligió desde la aplicación de su celular.

3. Pantalla de Visualización del Historial (Auditoría de Eventos)

Esta sección está pensada para que el usuario pueda revisar el registro de seguridad de su mochila y saber si fue manipulada en el pasado.

El usuario accede al apartado de Historial desde el menú.

La aplicación se conectará automáticamente a la base de datos de la mochila a través de internet.

En la pantalla se renderizará una lista ordenada cronológicamente con la fecha y hora exacta de cada suceso relevante.

Cada vez que pasa algo en el circuito físico de la mochila (por ejemplo, cuando el acelerómetro registra un sacudón, o cuando el zumbador se dispara por una apertura), el ESP32 envía un reporte automático a internet a través del servidor central.

Estos reportes quedan guardados de forma permanente en la base de datos del sistema. De esta manera, aunque la mochila se quede sin batería o se apague, el usuario siempre podrá ver en el historial de su celular el momento exacto en que alguien intentó tocar sus pertenencias

3 Conclusiones

Durante el desarrollo del trabajo hubo problemas que resolver, destacando: Persistencia Aislada ante Desconexión: Al delegar el historial a una base de datos externa como InfluxDB a través de Docker, se previene la pérdida de datos en caso de que el ESP32 se reinicie o se quede sin energía. Los eventos críticos ya fueron despachados y persistidos de manera segura en el servidor, permitiendo un análisis forense del estado del dispositivo incluso si este se encuentra fuera de línea (*Offline*).

Estos desafíos del desarrollo de este proyecto nos demostraron que el diseño de sistemas IoT reales va mucho más allá de conectar componentes en una placa de pruebas. La principal lección aprendida radica en la importancia del desacoplamiento. Diseñar aplicaciones asincrónicas coordinadas mediante hilos y sistemas embebidos gobernados por un FRTOS requiere una sincronización rigurosa basada en primitivas de comunicación seguras (como semáforos, banderas y colas). Aprendimos que agregar líneas de código extras sin protección en entornos concurrentes suele complicar la estabilidad del software; las soluciones robustas se logran mediante flujos simples, encapsulados y atómicos que respetan la exclusión mutua de los recursos compartidos.

4 Referencias

1. Silberschatz, A., Galvin, P. B., Gagne, G.: Operating System Concepts, 10th Edition. Wiley (2018)
2. Espressif Systems: ESP32-C3 Technical Reference Manual. v1.1 (2022)
3. Banks, A., Gupta, R.: MQTT Version 3.1.1. OASIS Standard (2014)
4. Foster, I., Kesselman, C.: The Grid: Blueprint for a New Computing Infrastructure. Morgan Kaufmann, San Francisco (1999)